MANUAL TÉCNICO

Sistema de Gestión de Emergencias Médicas

Liliana Carolina García Agudelo | IU Digital de Antioquia

1. Propósito y alcance

El proyecto implementa una simulación de un centro de emergencias médicas. Permite registrar incidentes, ordenarlos por prioridad, asignar ambulancias y médicos disponibles, simular la atención y consultar el estado del sistema desde la consola.

La solución se desarrolló en Java 25 con Maven. Los operadores generan llamadas de prueba; el despachador toma los casos de una cola concurrente; los equipos ejecutan la atención en hilos independientes; y el monitor presenta los estados y las estadísticas.

El alcance es académico y simulado. No se conecta con servicios de emergencia reales, geolocalización, hospitales ni redes externas. Las distancias y los tiempos representan datos de prueba.

2. Componentes del sistema

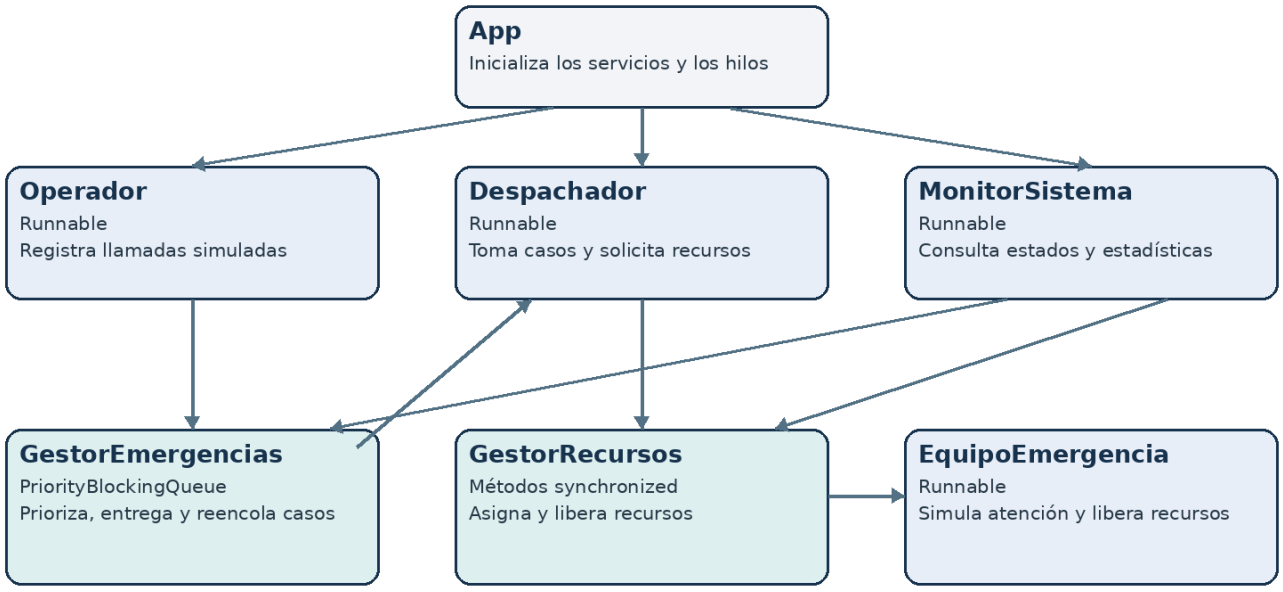


Figura 1. Relación general entre los componentes principales. Elaboración propia.

App inicia los servicios y los hilos. GestorEmergencias mantiene la cola de prioridad. GestorRecursos administra los recursos compartidos. Despachador coordina la asignación y EquipoEmergencia representa la atención. MonitorSistema consulta periódicamente la información disponible.

3. Organización de clases

El código se separa por responsabilidades. Esta distribución facilita ubicar las reglas del dominio, la administración de recursos y las tareas que se ejecutan en paralelo.

Paquete / clase	Responsabilidad
modelo: Emergencia	Datos del incidente: identificador, descripción, gravedad, estado, distancia y fecha de registro.
modelo: Ambulancia / Medico	Representan los recursos y su disponibilidad.
modelo: NivelGravedad / EstadoEmergencia / EstadoAmbulancia	Enumeraciones que limitan los valores de gravedad y estado.
modelo: AsignacionRecursos	Vincula una ambulancia y un médico asignados.
servicio: GestorEmergencias	Registra, prioriza, extrae y devuelve emergencias; conserva las estadísticas.
servicio: GestorRecursos	Registra recursos, realiza asignaciones y libera los recursos específicos.
servicio: EstadisticasSistema	Mantiene contadores y calcula el promedio de atención.
concurrencia: Operador / Despachador	Simulan la recepción de llamadas y la asignación de recursos.
concurrencia: EquipoEmergencia / MonitorSistema	Simulan la atención y presentan el estado del sistema.

4. Priorización de incidentes

La cola utiliza PriorityBlockingQueue y un comparador. Primero se ordena por gravedad: crítico, grave, moderado y leve. En igualdad de gravedad se compara la fecha de registro y, como último criterio, la distancia registrada.

La cola es concurrente y permite que varios hilos inserten y retiren elementos de forma segura. El método take() bloquea al despachador mientras no haya casos disponibles; cuando llega una emergencia, el hilo puede continuar sin consultar repetidamente la cola.

La distancia se conserva como criterio secundario final. En una ampliación sería conveniente revisar la regla de desempate para que el peso de la distancia y del tiempo de espera responda explícitamente a la política de despacho definida para el caso.

5. Concurrencia y sincronización

Las tareas que deben avanzar de manera independiente se implementan con Runnable y se ejecutan mediante Thread. El operador registra llamadas, el despachador retira casos, cada equipo simula la atención y el monitor consulta los datos en intervalos regulares.

Mecanismo	Uso en el proyecto	Propósito
PriorityBlockingQueue	Cola de emergencias	Permite insertar y extraer incidentes de forma segura entre hilos.
synchronized	Métodos de GestorRecursos	Protege la selección, asignación y liberación de ambulancias y médicos.
AtomicInteger	Identificadores y contadores	Evita actualizaciones perdidas en incrementos concurrentes.
AtomicLong	Acumulado de tiempo	Permite sumar duraciones desde distintos equipos.
Thread.interrupt()	Interrupción de la atención	Permite señalar la cancelación de espera o trabajo de un hilo y ejecutar la recuperación prevista.
PriorityBlockingQueue.take()	Comunicación despachador-cola	Mantiene al despachador esperando hasta que exista una emergencia.

6. Asignación y recuperación

GestorRecursos realiza la búsqueda de una ambulancia DISPONIBLE y un médico disponible dentro de una operación sincronizada. Cuando encuentra ambos, cambia sus estados y devuelve un objeto AsignacionRecursos. El equipo conserva esa referencia y, al terminar o ser interrumpido, libera exactamente esos recursos.

Si no hay recursos disponibles, el despachador devuelve la emergencia a la cola y espera antes de volver a intentarlo. Si el hilo del equipo se interrumpe durante la simulación, el bloque finally libera los recursos y la emergencia se reencola para un nuevo intento.

La sincronización de la asignación reduce el riesgo de que dos despachadores obtengan el mismo recurso. Mantener una única operación crítica sobre la lista de recursos también reduce la posibilidad de interbloqueo; no se realizan esperas de atención mientras se conserva el bloqueo del gestor.

7. Límites de la tolerancia a fallos

La recuperación implementada cubre la interrupción de un hilo durante la atención y la indisponibilidad temporal de recursos. No existe todavía una conexión de red ni un servicio externo cuya caída pueda simularse; por tanto, no se debe afirmar que el prototipo recupera desconexiones reales. La persistencia ante el cierre inesperado del proceso tampoco está implementada.

8. Pruebas y análisis de rendimiento

La validación se realiza con Maven y JUnit. Se probaron el registro y la prioridad, la devolución a la cola, la asignación y liberación de recursos, la indisponibilidad de recursos, la competencia entre 20 hilos y la recuperación después de interrumpir un equipo.

Nivel de carga	Condición evaluada	Resultado / lectura
Baja	Una emergencia y un conjunto de recursos disponibles.	La asignación se realiza y los recursos se liberan para reutilización. No se registró un tiempo de respuesta controlado.
Media	Operador genera llamadas periódicas; simulación con varios recursos.	En una captura: 17 registradas, 15 finalizadas y 0 pendientes; promedio de atención cercano a 8,01 s. Es una lectura puntual.
Alta / contención	20 hilos compiten por una ambulancia y un médico.	La prueba verifica una sola asignación exitosa; los demás hilos no obtienen el recurso ocupado.
Indisponibilidad	No hay ambulancia o no hay médico libre.	El gestor no crea una asignación incompleta; el despachador devuelve el incidente a la cola.
Interrupción	Se interrumpe un equipo durante la atención.	La prueba verifica la liberación de los recursos y el retorno de la emergencia a la cola.

En la salida de consola compartida durante una ejecución se observaron 17 emergencias registradas, 15 finalizadas, ninguna pendiente y un promedio de atención cercano a 8,01 segundos. En otra ejecución se observaron 7 registradas, 5 finalizadas y un promedio de 8,06 segundos. Son capturas puntuales de la simulación, no un benchmark controlado: los casos restantes podían estar en atención en el momento de la lectura.

No se midieron tiempos de respuesta desde la llamada hasta la llegada de la ambulancia, consumo de CPU, memoria ni escalabilidad con múltiples configuraciones de recursos. Para comparar cargas de forma rigurosa se requieren ejecuciones repetidas, cantidades controladas de llamadas y recursos, y registro automático de tiempos de espera y duración de atención.

9. Conclusiones y lecciones aprendidas

El prototipo muestra cómo varias tareas comparten una cola de emergencias y recursos limitados. La cola concurrente, los métodos sincronizados y las asignaciones explícitas ayudan a evitar asignaciones duplicadas y a mantener los estados de los recursos.

Las pruebas concurrentes complementan las unitarias: un método puede funcionar en secuencia y fallar cuando varios hilos compiten por el mismo recurso. Una interrupción también debe tratarse para no dejar recursos ocupados ni perder el incidente.

Como trabajo futuro se recomienda incorporar medición formal del tiempo de respuesta, una política cuantificada de gravedad/espera/distancia, persistencia de datos, gestión de fallos de conexión y cierre ordenado de los hilos.

10. Datos del proyecto

Responsable: Liliana Carolina García Agudelo. Desarrollo individual: análisis del caso, diseño, implementación, pruebas y documentación.

Repositorio: https://github.com/carolinag8916-droid/sistema-emergencias-medicas

Ejecución: desde la raíz del proyecto, ejecutar mvn clean test para validar y mvn exec:java "-Dexec.mainClass=com.emergencias.App" para iniciar la simulación. Se requiere JDK 25 y Maven configurado.

Referencia: Caso de estudio - Sistema de Gestión de Emergencias Médicas, material académico suministrado para la actividad de IU Digital de Antioquia.